Intern Assignment: skribbl.io Clone

Overview

Build an **end-to-end clone** of skribbl.io – a free online multiplayer drawing and guessing (pictionary) game.

How it works:

- Players join a room (public or private)
- Each round, one player draws a chosen word; others try to guess it
- Guessing correctly earns points; the player with the most points at the end wins
- Rounds rotate; everyone gets a turn to draw

Core Requirements

1. **Multiplayer rooms** – Create or join rooms; support public and private rooms
2. **Turn-based drawing** – One drawer per round; others guess
3. **Real-time drawing** – Canvas updates in real time for all viewers via WebSockets
4. **Word system** – Drawer picks a word from a list; others see hints or blanks
5. **Scoring** – Points for correct guesses; leaderboard; winner at game end
6. **WebSockets** – Real-time sync for drawing, guesses, chat, and game state

---

Feature Breakdown

Room & Lobby

Feature	Description
Create room	Host creates room with settings (player count, rounds, draw time, etc.)
Join room	Via link or room code
Lobby	Show players, ready-up, host starts game
Private room	Invite-only via link
Public room	Join random/open rooms (optional)

Game Flow

Feature	Description
Word selection	Drawer picks 1 of N words (e.g., 1–5 words)
Drawing	Drawer draws on canvas; strokes sync to all players in real time
Guessing	Others type guesses in chat; first correct guess gets points
Hints	Optional: reveal letters over time (e.g., _ _ a _ _)
Round end	Time runs out or word guessed; next drawer; next round
Game end	All rounds done; show winner and leaderboard

Drawing Tools

Feature	Description
Brush / Pen	Basic drawing with color
Colors	Multiple colors to choose from
Brush size	Thin / thick
Eraser	Clear strokes
Undo	Undo last stroke
Clear canvas	Clear entire canvas (drawer only)

Chat & Guessing

Feature	Description
Guess input	Type guess; send to server; check against word
Chat	General messages (drawer sees after round, or all can chat)
Correct guess notification	“PlayerX guessed the word!”
Hint display	Show revealed letters (e.g., after 30s, 60s)

Room Settings (host-configurable)

Setting	Options
Max players	2–20
Rounds	2–10
Draw time	15–240 seconds

Word count	1–5 words to choose from per round
Hints	0–5 hints, or disabled
Word mode	Normal, Hidden (no word shown), Combination (optional)

Tech Stack

Layer	Technology	Purpose
Frontend	React / Next.js / Vue	UI, canvas, lobby, game screens
Canvas	HTML5 Canvas API or Fabric.js / Konva.js	Drawing; sync stroke data
Backend	Node.js (Express) or Python (FastAPI)	API, game logic, WebSocket server
Real-time	**Socket.IO** or **ws**	Drawing sync, guesses, chat, game state
Words	JSON/DB word list	Categorized words (e.g., animals, objects, actions)

Recommended Stack (example)

- **Frontend:** React + TypeScript + Vite
- **Canvas:** HTML5 Canvas + custom drawing logic (or Fabric.js)
- **Backend:** Node.js + Express
- **WebSockets:** Socket.IO

WebSocket Events (Suggested)

Room & Lobby

Event	Direction	Payload	Description
`create_room`	Client → Server	`{ hostName, settings }`	Host creates room
`join_room`	Client → Server	`{ roomId, playerName }`	Player joins room

`player_joined`	Server → Clients	`{ player, players }`	Broadcast new player
`player_left`	Server → Clients	`{ playerId, players }`	Broadcast player left
`start_game`	Client → Server	`{}`	Host starts game; **Host only**

Game State

Event	Direction	Payload	Description
`game_state`	Server → Clients	`{ phase, round, drawerId, word, hints }`	Current game state
`round_start`	Server → Clients	`{ drawerId, wordOptions, drawTime }`	New round; drawer gets words
`word_chosen`	Client → Server	`{ word }`	Drawer chose word; server starts timer
`round_end`	Server → Clients	`{ word, scores, nextDrawer }`	Round over; show word, update scores
`game_over`	Server → Clients	`{ winner, leaderboard }`	Game finished

Drawing

Event	Direction	Payload	Description
`draw_start`	Client → Server	`{ x, y, color, size }`	Drawer starts stroke
`draw_move`	Client → Server	`{ x, y }`	Drawer continues stroke
`draw_end`	Client → Server	`{}`	Drawer ends stroke
`draw_data`	Server → Clients	`{ ...stroke }`	Broadcast stroke to all (including drawer for consistency)
`canvas_clear`	Client → Server	`{}`	Drawer clears canvas; broadcast
`draw_undo`	Client → Server	`{}`	Drawer undos last stroke; broadcast

Chat & Guessing

Event	Direction	Payload	Description
`guess`	Client → Server	`{ text }`	Player sends guess
`guess_result`	Server → Clients	`{ correct, playerId, playerName, points }`	Guess correct/incorrect
`chat`	Client → Server	`{ text }`	Chat message
`chat_message`	Server → Clients	`{ playerId, playerName, text }`	Broadcast chat

Functional Requirements Checklist

Must Have

- [ ] Create room with configurable settings
- [ ] Join room via link or code
- [ ] Lobby with player list; host starts game
- [ ] Turn-based rounds: one drawer, others guess
- [ ] Real-time drawing sync (strokes visible to all)
- [ ] Word selection for drawer (1–5 choices)
- [ ] Guessing: type word, get points for correct guess
- [ ] Scoring and leaderboard
- [ ] Game end with winner
- [ ] Basic drawing tools: brush, colors, undo, clear

Should Have

- [ ] Hints (reveal letters over time)
- [ ] Chat (guesses + optional general chat)
- [ ] Draw time countdown
- [ ] Private rooms (invite link)

Nice to Have

- [ ] Word categories (animals, objects, etc.)
- [ ] Eraser tool
- [ ] Kick/ban (host moderation)
- [ ] Votekick
- [ ] Multiple languages for word list

Deployment

Deploy the application so it is **publicly accessible** and fully working. Use one of:

Platform	Notes
Render	Full-stack, WebSocket support
Railway	Full-stack, WebSockets, easy setup
Vercel	Frontend + serverless; pair with separate WebSocket backend (e.g., Render, Railway)
Netlify	Frontend; use separate backend for WebSockets

- Provide a **live URL** (e.g., `https://your-skribbl-clone.onrender.com`)
- Core flow (create room → draw → guess → score) must work in production
- Include the live URL in README

Code Understanding

Be prepared to explain:

- How drawing strokes are captured, sent, and rendered across clients
- How game state (rounds, turn order, scoring) is managed
- How WebSockets are used for real-time sync
- Word-matching logic (case, trim, partial)
- Deployment setup and any platform constraints (e.g., WebSocket support on Vercel/Netlify)

Deliverables

1. **Working application** – End-to-end clone running locally and deployed publicly
2. **README.md** – Setup instructions + live deployment URL
3. **Architecture overview** – How WebSockets, canvas, and game logic integrate
4. **Code walkthrough readiness** – Ability to explain tech choices and flow

---

Bonus Ideas

- **OOP concepts for WebSocket servers** – Structure your WebSocket server using OOP: e.g., `Room` class (participants, game state, broadcast), `Game` class (rounds, scoring, turn logic), `Player` class, `MessageHandler` class, etc. Encapsulate game and room logic in well-defined classes.
- **Room settings** – Draw time, rounds, word count, hints (all configurable)

- **Word modes** – Normal, Hidden, Combination
- **Moderation** – Kick, ban, votekick, report
- **Custom word list** – Host can add custom words
- **Avatars** – Player avatars in lobby and game
- **Spectator mode** – Watch without playing
- **Replay** – Replay last round's drawing

Resources

- skribbl.io – Reference implementation
- Socket.IO Docs
- HTML5 Canvas API
- Fabric.js – Canvas library (optional)
- **Deployment:** Render | Vercel | Netlify | Railway